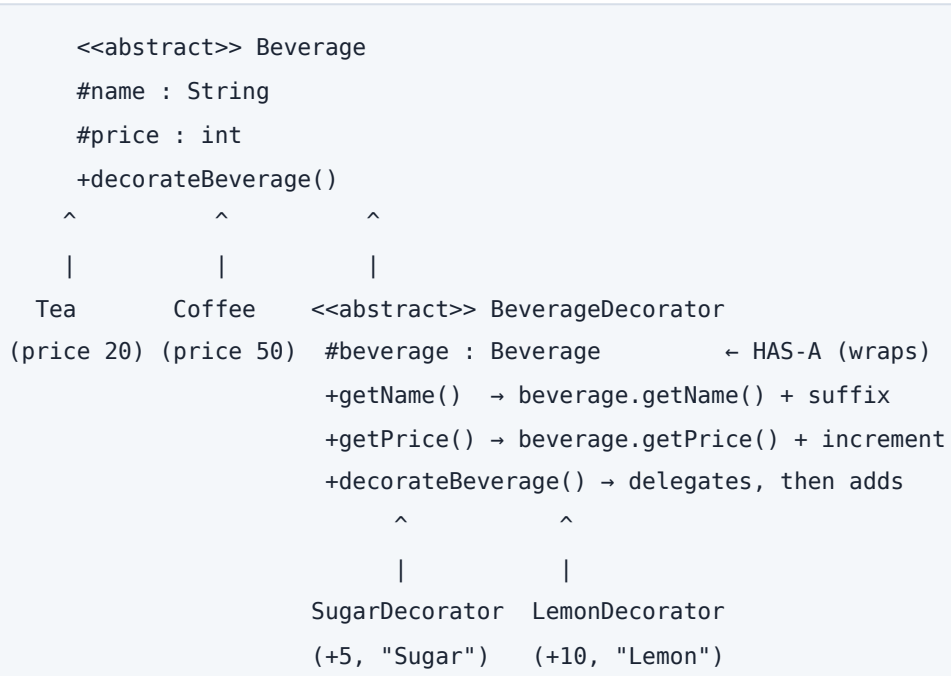
Decorator Design Pattern

> **GoF intent:** *Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.* A decorator **is-a** component (extends the same abstraction) and **has-a** component (wraps an instance), so wrappers can stack in any order and any depth at runtime — `new SugarDecorator(new LemonDecorator(new Tea("Assam Tea")))`.

Structure of this implementation



GoF role	Class in this module
Component (common abstraction)	`Beverage` — `name`, `price`, `decorateBeverage()`
ConcreteComponent (base objects)	`Tea` (20), `Coffee` (50)
Decorator (abstract wrapper)	`BeverageDecorator` — extends `Beverage`, holds a `Beverage`
ConcreteDecorator (add-ons)	`SugarDecorator` (+5), `LemonDecorator` (+10)
Client	`DesignPatternsApplication.main()`

How it works

1. `Tea` / `Coffee` are complete beverages on their own, each with a base price.
2. `BeverageDecorator` is the pattern's hinge: it **extends** `Beverage` (so a decorated drink is still a `Beverage` and can be re-wrapped) and **wraps** a `Beverage` instance.
3. Everything is computed **by delegation**, never copied:
4. `getName()` → wrapped name + `":"` + this decorator's suffix

5. `getPrice()` → wrapped price + this decorator's increment
6. `decorateBeverage()` → **first calls** **`beverage.decorateBeverage()`** (so the whole inner chain runs), then prints its own addition.
7. Stacking composes at runtime with no subclass explosion — adding `MilkDecorator` later means one new class, zero changes to `Tea` / `Coffee` (Open/Closed Principle). Subclassing every combination instead would need `TeaWithLemonAndSugar`, `CoffeeWithSugar`, … (2^n classes).

Verified output (`SugarDecorator(LemonDecorator(Tea/Coffee))`):

```
The cost of Assam Tea:20
Added Lemon to Assam Tea -> cost of Assam Tea:Lemon:30
Added Sugar to Assam Tea:Lemon -> cost of Assam Tea:Lemon:Sugar:35
The cost of Cappuccino:50
Added Lemon to Cappuccino -> cost of Cappuccino:Lemon:60
Added Sugar to Cappuccino:Lemon -> cost of Cappuccino:Lemon:Sugar:65
```

Every layer of the onion speaks in order — base beverage first, then each wrapper inside-out — because each decorator delegates before adding its own behavior. Prices: 20+10+5 = 35 ✓, 50+10+5 = 65 ✓.

Why this follows the pattern

- ✅ **Is-a + has-a:** `BeverageDecorator extends Beverage` *and* wraps a `Beverage` — the defining dual relationship of Decorator.
- ✅ **True delegation:** name, price, and behavior are all computed by forwarding to the wrapped object and augmenting the result — the decorated object is untouched.
- ✅ **Stackable & order-free:** any decorator accepts any `Beverage`, including an already-decorated one (verified above).
- ✅ **Open/Closed:** new toppings are new classes, not edits to existing components.
- ✅ Same shape as the classic real-world decorator: `new BufferedInputStream(new FileInputStream(f))` in `java.io`.

History

The first version accumulated name/price by copying state in the decorator constructor and never called `beverage.decorateBeverage()` — inner decorators' behavior was silently skipped ("Added Lemon…" never printed), and the demo in `main()` had the decorator lines commented out. Fixed on 2026-07-08: full delegation for `getName()` / `getPrice()` / `decorateBeverage()`, no overridable calls from constructors, and a live demo in `main()`.